

Sistemas de información

Alineación estratégica y ciclos de desarrollo de software

Ph. D. Jhon Alexander Garcia Camargo

10 de septiembre de 2026



Índice de contenidos

- Mapa de capacidades de negocio
- Indicadores de alineación
- Fundamentos del ciclo de vida de desarrollo de software (SDLC)
- Modelos de gestión

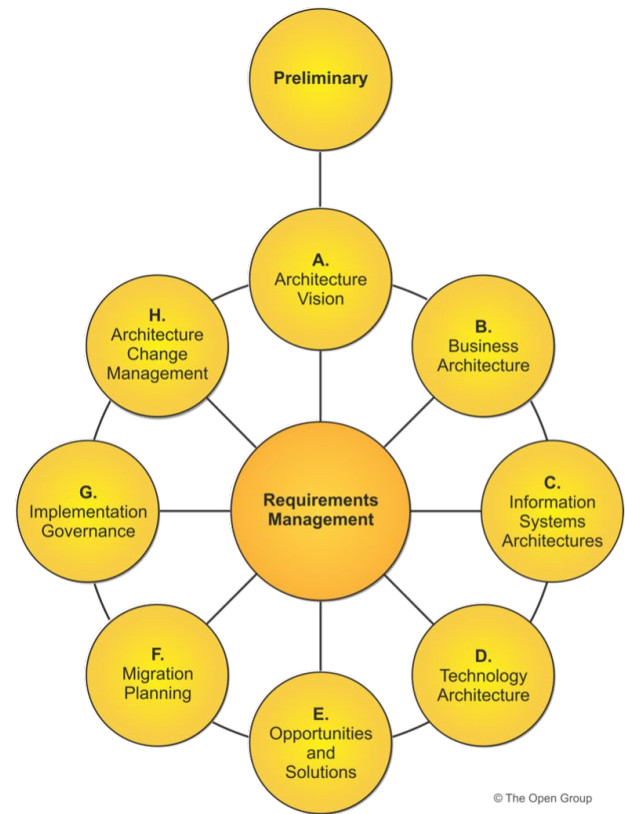


Mapa de capacidades de negocio

En TOGAF, una capacidad se define simplemente como la capacidad de hacer algo.

Una capacidad empresarial representa la capacidad de una empresa para hacer algo.

Para más información, visitar [Cómo liberar el potencial de su negocio: cómo crear un mapa de capacidades eficaz - Conexiam](#)



© The Open Group

Tomado de: [TOGAF | www.opengroup.org](http://www.opengroup.org)

Mapa de capacidades de negocio

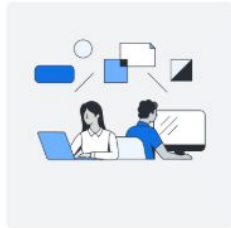
“Una capacidad empresarial es una habilidad particular que una empresa puede poseer o intercambiar para lograr un propósito o resultado empresarial concreto. Una capacidad empresarial describe lo que hace la empresa (resultados y niveles de servicio) que mejora la creación de valor para los clientes (...) una capacidad empresarial abstrae e integra a las personas, procesos/procedimientos, tecnología e información en los bloques esenciales necesarios para facilitar la mejora del rendimiento y el análisis de rediseño.” –

Ulrich Homann

"El mapeo de capacidades empresariales es el proceso de modelar lo que una empresa hace para alcanzar sus objetivos (sus capacidades), en lugar de cómo lo hace (sus procesos empresariales)." – Denise Cook

Mapa de capacidades de negocio

Scenarios for business capability maps



IT consolidation after a merger and acquisition



Strategic planning and IT investments



Product roadmap and development

Business capability map example
System Templates | January 22, 2020



Tomado de: <https://lucid.co/blog/a-quick-guide-to-business-capability-maps>

Opciones: [Las 10 mejores plantillas de mapeo de capacidad con ejemplos y muestras](#)

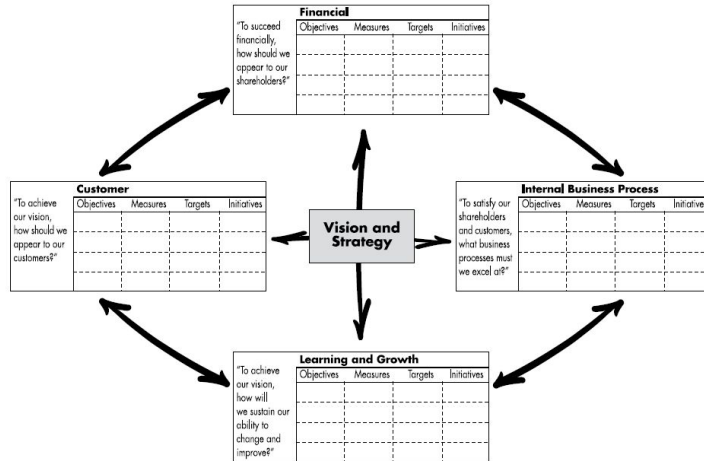
Indicadores de alineación



Tomado de: KRI vs KPI vs KCI Relationship b/w Key Performance, Risk & Control Indicators with examples solved

Cuadro de mando integral

Translating Vision and Strategy: Four Perspectives



Tomado de: [Kaplan Robert and Norton David, 2006. Using the Balanced Scorecard as a strategic management system](#)

Fundamentos del ciclo de vida de desarrollo de software (SDLC)

Intangible y moldeable

The intangible and malleable nature of software permits a wide variety of software development life cycle models, ranging from linear models in which the phases of software development are accomplished sequentially with feedback and iteration as needed followed by integration, testing, and delivery of a single product; to iterative models in which software is developed in increments of increasing functionality on iterative cycles; to agile models that typically involve frequent demonstrations of working software to a customer or user representative who directs development of the software in short iterative cycles that produce small increments of working, deliverable software. Incremental, iterative, and agile models can deliver early subsets of working software into the user environment, if desired.

Tomado de: [Guide to the Software Engineering Body of Knowledge Version 3.0, 2014. A Project of the IEEE Computer Society](#)

Fundamentos del ciclo de vida de desarrollo de software (SDLC)

Orden lógico, no cronológico

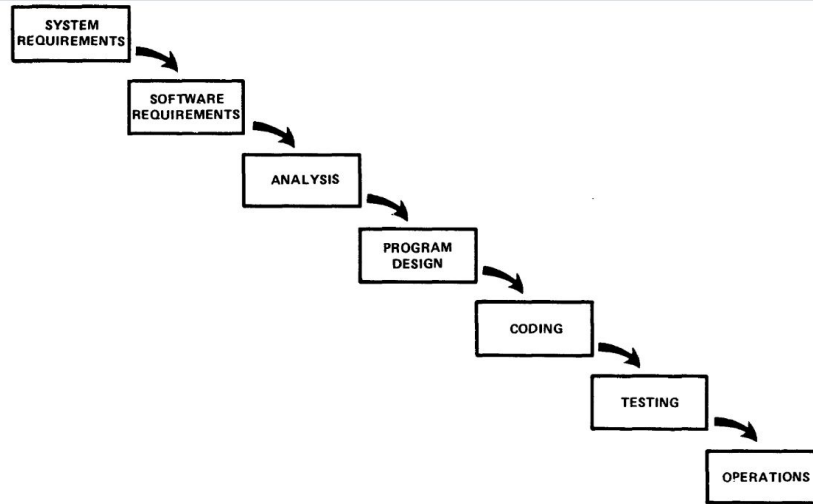


Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

Planificación: Justificación económica y factibilidad técnica (¿Podemos y debemos hacerlo?).

Análisis: El "QUÉ". Definición de requerimientos funcionales y no funcionales.

Diseño: El "CÓMO". Arquitectura de sistemas, diagramas de clases, diseño de base de datos.

Implementación: Codificación y configuración de infraestructura.

Pruebas (QA): Verificación frente a los requerimientos iniciales.

Mantenimiento: Corrección de errores y evolución (esta fase consume el 60-80% del costo total del ciclo de vida).

Tomado de: [Royce, W. W. \(1970\). Managing the Development of Large Software Systems.](#)

Sistemas de Información

Modelos de gestión

Predictivos

Cascada o su variante “V”

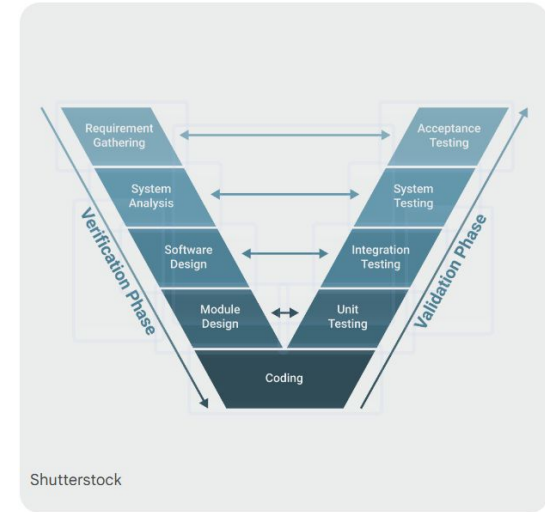
Se basan en la premisa de que los requerimientos son estables y bien conocidos desde el inicio.

Ejemplo: Software de control para una represa o un sistema de nómina con regulaciones legales fijas en Colombia

¿Cuándo usarlos?

Proyectos de alta criticidad (salud, aeroespacial) o donde el costo del cambio es extremo.

Tomado de: [Guide to the Software Engineering Body of Knowledge Version 3.0, 2014. A Project of the IEEE Computer Society](#)



Creado con Gemini 3

Modelos de gestión

Incrementales

Reconocen que los sistemas de información rara vez se entienden completamente al principio.

Modelo Espiral (Boehm): Se centra en el análisis de riesgos. Cada ciclo de la espiral incluye planificación, análisis de riesgos, ingeniería y evaluación.

Desarrollo Incremental: Se entrega una funcionalidad básica (Core) y se añaden "incrementos" en cada versión.

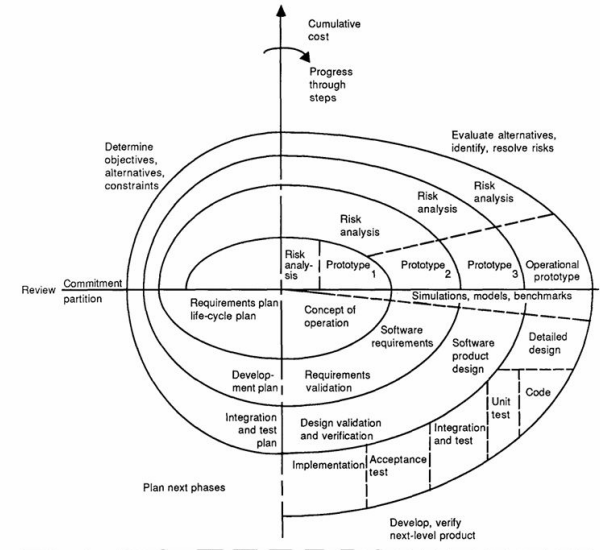


Figure 2. Spiral model of the software process

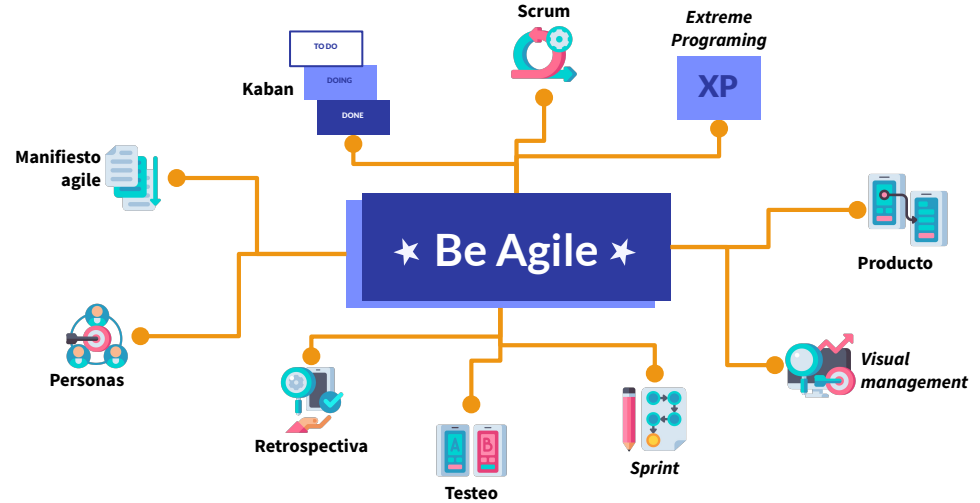
Tomado de: [Boehm, B. W. \(1988\). A Spiral Model of Software Development and Enhancement. IEEE Computer](#)

Adaptativos

Se abraza el cambio en lugar de intentar predecirlo.

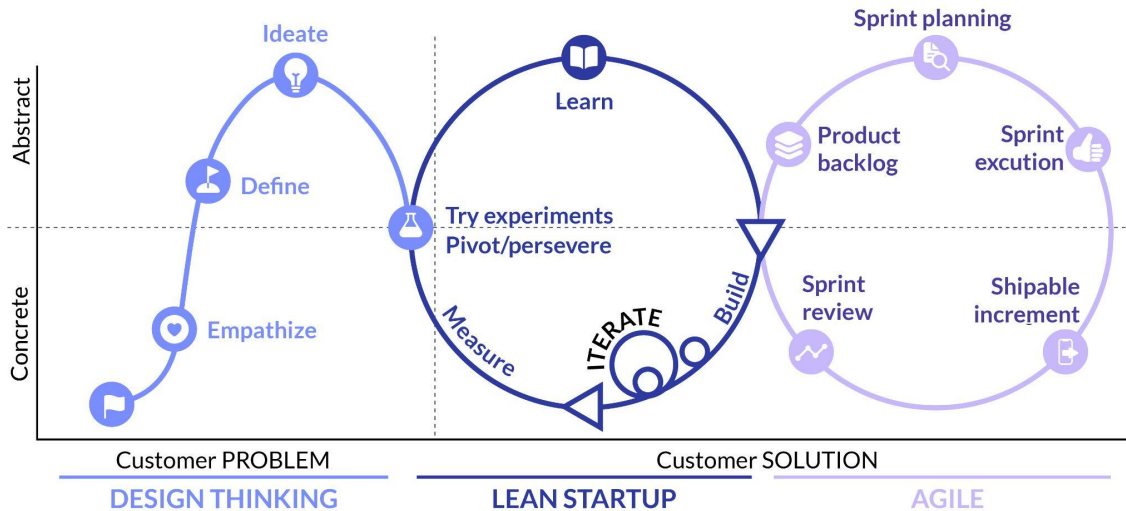
Scrum: Marco de trabajo basado en iteraciones cortas (Sprints), roles definidos y eventos de inspección.

Kanban: Gestión visual del flujo de trabajo, limitando el trabajo en curso (WIP) para evitar cuellos de botella.



Tomado de: [Schwaber, K., & Sutherland, J. \(2020\). La guía Scrum](#)

Marcos mixtos de gestión



Tomado de: [Design Thinking + Lean UX + Agile](#)





¡Gracias!



<https://orcid.org/0000-0002-4763-3345>



jhagarcia@unal.edu.co



3012778432